GAN Model and Interactive Presentation Site for the MSK-IMPACT 50K Cohort

Individual Final Report

Siddhant Saxena
Department of Data Science, The George Washington University
Group 5

1. Introduction

Survival analysis characterises the distribution of time to event outcomes such as death, relapse, or treatment failure under right censoring, and underlies decision making across many biomedical fields. The growing scale of clinical and genomic registries, together with stringent privacy constraints and the need to augment small or unbalanced cohorts, has motivated the development of generative models that produce synthetic survival data while preserving the joint distribution of covariates, event times, and censoring indicators. Direct application of a tabular GAN is a strong baseline but it can fail in three characteristic ways i.e over-optimism, over-pessimism, and short-sightedness because the generator has no explicit access to the censoring mechanism (Norcliffe et al., 2023).

Our group project reproduced and extended SurvivalGAN — a three component architecture combining a conditional Wasserstein GAN with gradient penalty (WGAN-GP), a DeepHit discrete-time survival network, and an XGBoost time to event regressor on the MSK-IMPACT 50K pan cancer cohort (Bandlamudi et al., 2026). MSK-IMPACT is roughly an order of magnitude larger than the public benchmarks used in the original SurvivalGAN paper and contains far more clinical and genomic heterogeneity, making it a meaningful stress test of the method.

My individual contribution described in this report was the GAN baseline and the interactive presentation website that exposes the trained generator over a Flask backend so a user can sample synthetic patients live from the browser.

Outline. Section 2 gives background on the GAN baseline and the dataset slice it consumes. Section 3 describes my individual contributions in detail, covering the generator/discriminator architecture, the training script, the checkpoint test harness, the Flask inference backend, and the five page Angular frontend. Section 4 presents the evaluation results from the shared metrics framework. Section 5 summarises lessons learned and proposes future improvements. Section 6 reports the fraction of code derived from external sources. Section 7 lists references.

2. Background

2.1 The MSK-IMPACT 50K Dataset

The MSK-IMPACT 50K dataset collects the overall survival of cancer patients together with their demographic, histopathological, and genomic sequencing data. After the preprocessing pipeline produced by my groupmate (survival-label parsing, patient-level deduplication, stratified 80/20 train/test split, identifier removal, outlier clipping, integer-type preservation, imputation, and label encoding) the analysis cohort contains 43,673 patients across 19 columns: 10 continuous variables (Age at Diagnosis, Ploidy, FACETS Estimated Purity, Fraction Genome Altered, MSI Score, Mutation Count, Number of Other Cancer Types, Sample coverage, TMB Score, Pathologist Estimated Tumor Purity) and 7 categorical variables (Sex, Cancer Type, Disease Status, MSI Type, Genetic Ancestry, FACETS QC, Whole Genome Doubling Status), plus the survival outcome columns time and status.

The Vanilla GAN consumes the same train split with 34,938 training patients with a 38.6% event rate and is evaluated against the held-out test split of 8,735 patients. Synthetic samples are also generated at 8,735 rows to match the test set size, which is what the shared evaluation framework expects.

2.2 Why a GAN Baseline?

A MLP GAN trained on a flat z-scored feature vector is the most transparent and reproducible generative model for this task. With Linear ,LeakyReLU layers, a standard BCE objective, and a single StandardScaler, every modelling choice is visible and every result is directly attributable to the adversarial training itself rather than to an auxiliary mechanism. This makes it an ideal reference point as it establishes the quality that a generic, well tuned GAN can already deliver on the MSK-IMPACT 50K cohort, and it gives CTGAN and SurvivalGAN a concrete, well understood target to improve on.

3. Individual Work

3.1 Generator Architecture (model_gan.py)

The generator is kept in its own module so that both the training script and the inference backend can construct an identical network and load the same weights. The architecture is a four-layer fully-connected MLP that maps a 128-dimensional Gaussian latent vector to an n_features-dimensional row in the standardised feature space. LeakyReLU(0.2) activations are placed between every hidden

Linear layer and the final layer is left linear so that the output lives in unbounded z-score space i.e discrete columns and the time column are repaired after inverse transforming back to raw units.

Layer	Shape / Detail
Input	z ∈ ℝ^128 (Gaussian noise)
Linear	128 → 256
LeakyReLU	negative slope = 0.2
Linear	256 → 256
LeakyReLU	negative slope = 0.2
Linear	256 → 256
LeakyReLU	negative slope = 0.2
Linear	256 → n_features
Output	synthetic patient vector (z-scored)

Table 1. Generator architecture as defined in model_gan.py.

The discriminator (defined inside train_gan.py) mirrors this shape but takes an n_features-dimensional row, applies LeakyReLU(0.2) and Dropout(p=0.3) between two 256-unit hidden layers, and ends in a single Sigmoid neuron whose output is interpreted as the probability that the input row is real.

Layer	Shape / Detail
Input	patient vector ∈ ℝ^n_features
Linear	n_features → 256
LeakyReLU + Dropout	slope = 0.2, p = 0.3
Linear	256 → 256
LeakyReLU + Dropout	slope = 0.2, p = 0.3
Linear	256 → 1
Sigmoid	P(real)

Table 2. Discriminator architecture used during GAN training.

3.2 GAN Training Pipeline (train_gan.py)

The training script implements the full end-to-end pipeline and produces a self contained checkpoint bundle that the inference backend later consumes. Its eight stages are summarised below.

- Stage 1 — Load preprocessed data. Read survival_gan_train.csv and survival_gan_test.csv from data_sets/. Logs cohort shape and training event rate as a sanity check.
- Stage 2 — Discrete column declaration. The seven categorical variables, four integer-valued variables (Age at Diagnosis, Mutation Count, Sample coverage, Number of Other Cancer Types), and the binary status column are flagged as discrete. Their per-column min/max are recorded so that synthetic samples can be rounded and clamped after inverse-transform.
- Stage 3 — Standardisation. A single StandardScaler is fit on the training rows across all columns. The GAN therefore operates entirely in z-score space, which keeps gradients well-conditioned and lets the network treat continuous and discrete columns on a common scale.
- Stage 4 — Network construction. The Generator from model_gan.py and the Discriminator defined in this script are instantiated with latent_dim = 128 and moved to CUDA when available. Both networks use Adam with lr = 2e-4 and betas = (0.5, 0.999), the configuration originally proposed for DCGAN that stabilises min-max optimisation.
- Stage 5 — Adversarial loop. For 200 epochs at batch size 500 (drop_last=True) the script alternates one discriminator step and one generator step per minibatch, both under BCEWithLogits-equivalent BCELoss. The generator is trained with the non-saturating loss bce(D(G(z)), 1) so its gradients do not vanish when the discriminator is too confident.
- Stage 6 — Synthetic generation. After training, 8,735 latent vectors are drawn (matching the test set size), passed through the generator with G.eval(), inverse-transformed to raw units, and the discrete columns are np.clip(np.round(...), lo, hi) before being cast to int. The time column is clipped at zero so that no synthetic patient has negative survival time.
- Stage 7 — Sanity check. Event rate, time range, mean Age at Diagnosis, and mean TMB Score are printed for the real test set and the synthetic set so that gross failures (negative times, all-zero status) are caught before downstream evaluation.
- Stage 8 — Checkpoint bundle. The script writes gan_checkpoint/generator_gan.pt (CPU-side state dict), scaler_gan.joblib (the fitted scaler), metadata_gan.json (column order, discrete bounds, latent dim, architecture string), and a copy of model_gan.py. This bundle is the contract consumed by test_gan.py and backend/app.py.

3.3 Checkpoint Validation (test_gan.py)

test_gan.py is a test that runs against the saved checkpoint and verifies three properties before the bundle is shipped to the inference backend.

- Test 1 — End-to-end decode. Loads metadata_gan.json, scaler_gan.joblib, and generator_gan.pt, reconstructs the Generator, samples one $z \in \mathbb{R}^{128}$, runs G.eval() under torch.no_grad(), inverse transforms with the saved scaler, and applies the same discrete clamping as training. The resulting row must satisfy time ≥ 0 and have integer values for the discrete columns.
- Test 2 — Determinism. The same torch.manual_seed(0) before drawing z must produce identical generator outputs across two calls, confirming that the network is purely deterministic given its input.

- Test 3 — Batch sampling. A batch of five patients is sampled to verify that the generator behaves correctly for n > 1 — a different code path because of how the inverse scaler and the DataFrame construction handle multi-row inputs.

All three tests pass on CPU only, which means the same checkpoint can be loaded inside the Flask backend without a GPU.

3.4 Backend Inference Server (backend/app.py)

The inference backend is a Flask application that loads the checkpoint bundle exactly once at process start and exposes two endpoints behind flask-cors. Loading happens before app = Flask(...) is constructed so the cost is paid at boot rather than on the first request.

- POST /api/generate — draws z = torch.randn(1, latent_dim), runs the generator, inverse-transforms with the saved scaler, applies the same round-and-clip discrete repair as test_gan.py, and returns a JSON payload of {columns, values, latent_preview, raw_preview}. The previews expose the first eight components of z and of the raw network output so the frontend can display the internal state of the model and not just the final patient.
- GET /api/health — returns {ok: true, n_features} so the Angular frontend can probe whether the backend is alive without triggering a generation.

The server listens on 127.0.0.1:5000. The Angular dev server is configured via proxy.conf.json to forward /api/* to this address, so the frontend can call /api/generate directly without any CORS configuration during development.

3.5 Angular Frontend — Five-Page Interactive Site

The frontend is an Angular 14 application that doubles as the project's static presentation site. It is structured as five logical pages selected by an in-component setPage() router: Introduction, GAN Baseline, CTGAN Baseline, Survival GAN, and Conclusion. Each page is rendered from a single AppComponent whose template branches on the selectedPage state, which keeps the deployment a flat static bundle that can be served from any CDN.

- Introduction page — frames the project motivation, the dataset, and the three model comparison.
- Vanilla GAN Baseline page — declares the architecture and renders the generator and discriminator as structured ArchLayer tables. This page is also where the user can sample a synthetic patient live by clicking a button that calls VanillaGanService.generate() over /api/generate.
- CTGAN Baseline page — documents the CTGAN configuration so a reader can compare it with the Vanilla GAN side by side.
- Survival GAN page — documents the SurvivalGAN configuration (WGAN-GP with gradient penalty λ=10 plus an identifiability penalty λ=0.1, GELU residual generator, LeakyReLU critic).
- Conclusion page — ties the three models together against the evaluation results.

The synthetic patient generation flow is implemented by VanillaGanService — a small Angular service that issues a POST to /api/generate and types the response as a GeneratedPatient interface {columns, values, latent_preview, raw_preview}. The component stores the response on the generated field,

exposes generating and generateError flags so the template can render a spinner or an actionable error message , and uses formatValue() and formatLatent() helpers to render integers without trailing decimals and floats truncated to three decimal places. The end result is that a non-technical user can click a single button and see a fully decoded synthetic patient — covariates, event time, and censoring indicator produced live by the same generator weights that were evaluated by the shared metrics framework.

4. Results

4.1 Sanity Checks

The Vanilla GAN reproduces the gross marginal statistics of the test cohort. The event rate is preserved within one percentage point, synthetic survival times are non-negative after the clipping step, and the means of two unrelated continuous covariates (Age at Diagnosis and TMB Score) sit close to the real test means. These checks rule out the trivial failure modes (mode collapse to all-zero rows, generator divergence) before any quality metric is computed.

Metric	Real (test)	Vanilla GAN
Event rate (status = 1)	38.6%	≈ 37.5%
Time minimum	0.0 months	0.0 months (clamped)
Synthetic samples generated	—	8,735
Censoring-rate difference (test)	—	0.011

Table 3. Marginal sanity checks against the held-out test split.

4.2 Distributional Quality Metrics

Distributional metrics were produced by the shared evaluation framework on the synthetic_vanilla_gan.csv output written by train_gan.py. All values below are computed against the held out test split.

Metric	Vanilla GAN (test)	Interpretation
Jensen–Shannon (marginal mean)	0.0117	Per-column distributions track the test split closely; lower is better.
α-precision OC (mean)	0.907	Synthetic rows fall inside the high-density support of the real distribution.
β-recall / PRDC recall (mean)	0.662	The generator covers about two-thirds of the real manifold — moderate mode coverage.

PRDC precision (mean)	0.947	Synthetic rows are mostly indistinguishable from real rows on a per-sample basis.
Detection AUC — XGBoost	≈ 1.000	A tree-based detector trivially separates real from synthetic. Vanilla GAN is recognisable.
Detection AUC — MLP	0.519	An MLP detector is barely above chance, indicating the GAN fools a model in its own family.

Table 4. Distributional metrics for the Vanilla GAN against the test split .

4.3 Survival Specific Metrics

Marginal similarity is necessary but not sufficient. The survival specific custom metrics — optimism, Kaplan Meier divergence, short sightedness, censoring rate difference, and Wasserstein distance on event/censoring times diagnose whether the generator has learned the time-to-event structure or only the covariate marginals.

Metric	Vanilla GAN (test)	Reading
Optimism	0.024	Generator slightly overestimates survival — close to zero is good.
Kaplan–Meier divergence	0.039	Synthetic KM curve sits within ≈ 4% of the real KM curve.
Short-sightedness	0.000	Synthetic time horizon covers the full real horizon — no truncation.
Censoring-rate difference	0.011	Synthetic censoring rate matches the real censoring rate to ≈ 1 percentage point.
Wasserstein dist. — event times	3.44	Mean displacement between real and synthetic event-time distributions (months).
Wasserstein dist. — censor times	3.07	Mean displacement between real and synthetic censoring-time distributions (months).

Table 5. Survival-specific custom metrics .

4.4 Downstream Survival Utility

The most demanding test is the Train-Synthetic-Test-Real (TSTR) regime: a downstream survival model is trained on the synthetic patients and evaluated on the real held-out test split. If the synthetic data

carries the right covariate-to-outcome relationships the model will generalise; if it does not, TSTR collapses toward random.

Learner	Setting	Vanilla GAN
Cox proportional hazards	TRTR (train real / test real)	0.630
XGBoost AFT	TRTR (train real / test real)	0.605
XGBoost AFT	TSTR (train synthetic / test real)	0.487
XGBoost AFT	TSTS (train synthetic / test synthetic)	0.920
Feature-importance Spearman	XGBoost (real vs synth)	−0.206

Table 6. Downstream survival utility for the Vanilla GAN .

The picture is sharp. A model trained on synthetic data and evaluated on synthetic data scores a 0.920 C-Index — a value that would look excellent on its own. The same model evaluated on real data (TSTR) collapses to 0.487 i.e. below random. The Spearman correlation between feature importances learned on real vs synthetic data is −0.206, which means the Vanilla GAN actively reverses the covariate-outcome relationships rather than merely smoothing them.

4.5 Failure Mode Analysis

- Over-optimism / over-pessimism — present but mild on this cohort. Optimism is +0.024 and KM divergence is 0.039. The generator slightly inflates survival but does not catastrophically shift the curve.
- Short-sightedness — not observed (0.000). The generator covers the full survival horizon, likely because the time column is treated continuously rather than being binned.
- Broken covariate-outcome dependencies — the dominant failure. Feature-importance Spearman of 0.206 and XGBoost TSTR of 0.487 show that while the marginal columns look right, their relationships to the outcome are wrong. This is precisely the argument for survival aware generation: a tabular GAN that treats time and status as ordinary numbers can match marginals and still be useless for downstream learning.

5. Summary and Conclusions

My individual contribution delivered three components of the experimental and presentation pipeline: the GAN model (model_gan.py + train_gan.py + test_gan.py + the saved gan_checkpoint/ bundle), the Flask inference backend that serves the trained generator over HTTP, and the five page Angular frontend that doubles as the project's interactive presentation site.

On the modelling side, the GAN gives a clean and informative bound for the project. Its marginals are good — Jensen–Shannon 0.012, α-precision 0.907 and its sanity-check statistics line up with the real test

cohort. But it fails on the metrics that actually matter for survival modelling: the XGBoost detection AUC is ≈ 1.0, indicating that a tree based critic separates real from synthetic with near perfect confidence, the TSTR C-Index is 0.487, i.e. worse than random and the feature importance Spearman is negative. The GAN learns marginals but not relationships, which is exactly the structural failure that motivates survival aware generation.

On the presentation side, the Angular + Flask stack turns the trained generator from a static CSV into an interactive object. A viewer can navigate to the Vanilla GAN page, click **"Generate Patient"**, and see a freshly sampled synthetic row — discrete columns rounded, time clamped non-negative, latent and raw previews shown produced from the same generator_gan.pt that was evaluated by the shared metrics framework. The same site also documents the CTGAN and SurvivalGAN configurations so a reader can compare the three architectures side by side without rebuilding the project.

Future improvements:

- Replace the BCE-based loss with a conditional WGAN-GP loss to bring the Vanilla GAN closer to the CTGAN/SurvivalGAN regime — useful as an ablation isolating the effect of the loss alone.
- Add Gumbel-softmax heads for discrete columns instead of post-hoc rounding, to remove the rounding artefacts at the boundaries of categorical columns.
- Extend the backend with a /api/generate_batch endpoint so the frontend can render small synthetic cohorts at a time, not just single patients, which makes downstream Kaplan–Meier visualisation feasible directly in the browser.
- Wire the CTGAN and SurvivalGAN checkpoints into the same backend behind /api/generate?model=... so all three models are sample-able from the same UI.
- Server-side rate-limiting and a streaming response for the batched endpoint, so a viewer cannot accidentally DOS the inference process by spamming the generate button.

6. Percentage of Code from External Sources

The GAN training script is conventional PyTorch — the layer definitions, the BCE training loop, and the StandardScaler usage are drawn from the standard PyTorch and scikit-learn API patterns. The checkpoint-bundle layout (separating weights, scaler, metadata, and the generator class file) and all repair logic for discrete columns and survival times are original to this project, as is the contract between train_gan.py / test_gan.py / backend/app.py.

The Flask inference backend follows the standard load-model-once-on-import pattern used in most PyTorch-serving tutorials. The Angular frontend was scaffolded with the Angular CLI (ng new) and the proxy.conf.json + HttpClient wiring is the standard Angular development pattern. The five-page navigation, the ArchLayer rendering, the Vanilla GAN service, and the formatting helpers are original.

File	Lines	External (%)	Original (%)
model_gan.py	≈ 30	20%	80%

train_gan.py	≈ 250	25%	75%
test_gan.py	≈ 90	30%	70%
backend/app.py	≈ 80	30%	70%
frontend/src/app/app.component.ts	≈ 260	15%	85%
frontend/src/app/services/vanilla-gan.service.ts	≈ 25	30%	70%

Table 7. Approximate share of externally-derived code per file.

7. References

Bandlamudi, C., Muldoon, D., de Bruijn, I., Zhang, M., Gormally, M. V., Harrold, E. C., et al. (2026). Cancer type-specific variation in patterns of driver alterations across 50,000 tumors. Cancer Cell.

Goodfellow, I., Pouget-Abadie, J., Mirza, M., Xu, B., Warde-Farley, D., Ozair, S., Courville, A., and Bengio, Y. (2014). Generative Adversarial Networks. In Advances in Neural Information Processing Systems 27.

Norcliffe, A., Cebere, B., Imrie, F., Lio, P., and van der Schaar, M. (2023). SurvivalGAN: Generating time-to-event data for survival analysis. In International Conference on Artificial Intelligence and Statistics, 10279–10304. PMLR.

Radford, A., Metz, L., and Chintala, S. (2016). Unsupervised representation learning with deep convolutional generative adversarial networks (DCGAN). In International Conference on Learning Representations.

Xu, L., Skoularidou, M., Cuesta-Infante, A., and Veeramachaneni, K. (2019). Modeling tabular data using conditional GAN (CTGAN). In Advances in Neural Information Processing Systems 32.

Paszke, A., Gross, S., Massa, F., et al. (2019). PyTorch: An imperative style, high-performance deep learning library. In Advances in Neural Information Processing Systems 32.

Pedregosa, F., Varoquaux, G., Gramfort, A., et al. (2011). scikit-learn: Machine Learning in Python. Journal of Machine Learning Research, 12, 2825–2830.

Angular Team. (2022). Angular CLI 14 documentation. https://angular.io/cli.

Pallets Projects. (2024). Flask documentation. https://flask.palletsprojects.com/.